

SUMMER INTERNSHIP PROJECT 2026

DATA ANALYSIS WITH HOUSE SALES DATA

STUDENT NAME- SAPTAPARNA ROY

COURSE- BSC STATISTICS

BATCH: 2025- 2029

INSTITUTE NAME- ST. XAVIER'S COLLEGE
(AUTONOMOUS), KOLKATA

PERIOD OF INTERNSHIP- 18TH MAY, 2026 TO 14TH JUNE,
2026

REPORT SUBMITTED TO: IDEAS- INSTITUTE OF DATA
ENGINEERING, ANALYTICS, AND SCIENCE FOUNDATION,
ISI KOLKATA

1. ABSTRACT

This study analyses the house price india.csv dataset with 14,620 rows and 23 columns to identify factors affecting house prices. The data was loaded and verified, the shape was (14620, 23) and 0 duplicate rows were found. Data types were checked and missing values were inspected, then 7 columns including Date, Longitude, and Postal Code were dropped, leaving 16 columns. Descriptive statistics were generated for all numerical columns. Houses grouped by bedrooms showed average price increases from rupees 308631.77 for 1-bedroom houses to rupees 1204353.17 for 8-bedroom houses. Correlation of all numerical features

with Price was calculated, and 5 features had correlation greater than 0.5 which are number of bathrooms, living area, grade of the house, Area of the house (excluding basement), and living area renov. These 5 features were used as X and Price as y, with a train-test split of 0.3 and random state=123. The Linear Regression model achieved an R^2 score of 0.5627, meaning 56.27% of price variation was explained. For comparison, K-Means clustering was applied to MNIST with 10 clusters, giving a macro-averaged F1 score of 0.5100. A 10x10 confusion matrix was also generated to check cluster accuracy. The key finding is that living area, house grade, number of bathrooms, area excluding basement, and living area renovated are the strongest predictors of price in this dataset.

2. INTRODUCTION

Housing is one of the biggest expenses for any family, so predicting house prices is a useful problem to solve. This project uses the house price india.csv dataset which has details of 14,620 houses across India with 23 different features like area, bedrooms, grade, etc. The main purpose of this project is to understand which factors actually affect house prices and build a simple model to predict them. For this analysis, Python and Pandas were used for data cleaning and exploration. Scikit-learn was used to calculate correlations, split data, and train a Linear Regression model. Feature selection was done using correlation, and only features with correlation > 0.5 with Price were kept for modelling. Apart from house data, K-Means clustering was also tried on MNIST digits to practice unsupervised learning. The project shows how basic ML and data analysis can be applied to real estate data to find useful insights. Overall, the goal is to learn the full workflow from data cleaning to model building and evaluation.

The lists of topics that I have received training during the learning period of my internship includes basics of Python, Pandas which was used for data handling and cleaning, Matplotlib for plots and graphs, basic data analysis, Linear Regression for prediction, and K-Means clustering for MNIST were also done to cover ML concepts.

3. PROJECT OBJECTIVE

- Understanding the house price factors: Finding out which features like living area, grade, and bathrooms actually affect house prices in the dataset.
- Practice data cleaning: Learn to load data with Pandas, check for missing values/duplicates, and drop irrelevant columns.
- Apply data analysis: Use descriptive stats and grouping to see trends, like how price changes with number of bedrooms.
- Building a prediction model: Train a Linear Regression model using Scikit-learn to predict house prices based on selected features.
- Learning ML basics: Try feature selection with correlation and also practice unsupervised learning using K-Means clustering on MNIST digits.

4. METHODOLOGY

1. Data Collection

The dataset used was house_price_india.csv. It had 14,620 rows and 23 columns with details like Price, bedrooms, living area, grade, etc. The data was loaded into Python using Pandas ie. `pd.read_csv()`. No survey or primary data collection was done for this project.

2. Data Cleaning

First the shape of the dataset was checked using `.shape` and it was found to be (14620, 23). Then duplicate rows were checked with `.duplicated().sum()` and 0 duplicates were found in total. Data types and missing values were checked using `.info()` and `.isnull().sum()`. After that, 7 columns like Date, Postal Code, Latitude, Longitude were dropped because they were not relevant for price prediction. This left 16 columns for further analysis.

3. Exploratory Data Analysis

Descriptive statistics were generated using `.describe()` to see mean, std, min, max and quartiles of the numerical columns. Data was grouped by Bedrooms to find average price for each of the bedroom groups.

Correlation of all numerical features with Price was calculated using `.corr()`.

4. Feature Selection and Data Splitting

From the correlation results, 5 features with correlation > 0.5 were selected: number of bathrooms, living area, grade of the house, area excluding basement, and living_area_renov. Price was set as the target variable y and the 5 features as X . The data was split into training and testing sets using `train_test_split` from Scikit-learn with test size=0.3 and random state=123. So, 70% data went for training and 30% for testing.

5. Model Selection and Training

For prediction, Linear Regression was chosen because Price is a continuous value and the relationship with selected features was linear in nature. The model was imported from `sklearn.linear_model.LinearRegression`. It was trained on X train and y train using `.fit()`. Predictions were made on X test using `.predict()`.

6. Model Validation

The model was evaluated using R^2 score from `sklearn.metrics.r2_score`. The R^2 score obtained was 0.5627, which means 56.27% of the variation in price was explained by the model. For the MNIST part, K-Means clustering with 10 clusters was used. Cluster labels were mapped to actual digits and validated using classification report and confusion matrix. The macro F1 score was 0.5100 which means that the model has moderate performance. It is the average F1 score across all the classes. Since it's close to 0.5, it shows the model struggles with prediction accuracy and needs improvement in handling the class imbalance.

7. Tools and Methods Used

Python was the main language. Pandas and NumPy were used for data handling. Matplotlib was used for plots and graphs (which was not required for answering the questions). Scikit-learn was used for `train_test_split`, Linear Regression, K-Means clustering and for displaying the confusion matrix.

5. DATA ANALYSIS AND RESULTS

Data Loading and Initial Checks

The dataset `house_price_india.csv` was loaded using Pandas `pd.read_csv()`. Initial inspection was done using `.head()` to view first 5 rows, `.shape` which showed 14,620 rows and 23 columns, and `.info()` to check data types and non-null counts. No missing values were present in the dataset.

Import `pandas` as `pd` and `numpy` as `np`. Download the data `house_price_india.csv` from <https://drive.google.com/drive/folders/1gieHICVDBbUKMZiSF4YRQMAJic-w50JM?usp=sharing> and load the dataset into a DataFrame called `house_data` and display its first 5 rows.

Expected Output: A table showing the first 5 rows of the house price dataset.

```
In [ ]: # Write your answer here
import pandas as pd
house_data = pd.read_csv("house_price_india.csv")
house_data.head()
```

Out []:

	id	Date	number of bedrooms	number of bathrooms	living area	lot area	number of floors	waterfront present	number of views	condition of the house	...	Built Year	Renovation Year	Postal Code	Latitude	Longitude
0	6762810145	42491	5	2.50	3650	9050	2.0	0	4	5	...	1921	0	122003	52.8645	-114.557
1	6762810635	42491	4	2.50	2920	4000	1.5	0	0	5	...	1909	0	122004	52.8878	-114.470
2	6762810998	42491	5	2.75	2910	9480	1.5	0	0	3	...	1939	0	122004	52.8852	-114.468
3	6762812605	42491	4	2.50	3310	42998	2.0	0	0	3	...	2001	0	122005	52.9532	-114.321
4	6762812919	42491	3	2.00	2710	4500	1.5	0	0	4	...	1929	0	122006	52.9047	-114.485

5 rows × 23 columns

Inspect the `house_data` DataFrame. Print its shape to see the number of rows and columns, and then check for the total number of duplicate rows in the dataset.

Expected Output: The shape of the DataFrame (e.g., `(14620, 23)`) and the total count of duplicate rows.

```
# Write your answer here
print(house_data.shape)
print(house_data.duplicated().sum())
```

```
(14620, 23)
0
```

Descriptive Statistics

Basic statistics were generated using `.describe()` for numerical columns. This gave mean, std, min, 25%, 50%, 75%, and max values for price, living area, bedrooms, bathrooms, etc. From this, it was observed that house prices had a wide range from minimum to maximum value.

Generate and display the descriptive statistics for only the numerical columns of the cleaned `house_data` DataFrame.

Expected Output: A table showing count, mean, std, min, 25%, 50%, 75%, and max for the numerical columns.

```
# Write your answer here
```

```
cleaned_data.describe()
```

	id	number of bedrooms	number of bathrooms	living area	lot area	number of floors	waterfront present	number of views	condition of the house	grade of the house	A house(t
count	1.462000e+04	14620.000000	14620.000000	14620.000000	1.462000e+04	14620.000000	14620.000000	14620.000000	14620.000000	14620.000000	146
mean	6.762821e+09	3.379343	2.129583	2098.262996	1.509328e+04	1.502360	0.007661	0.233105	3.430506	7.682421	18
std	6.237575e+03	0.938719	0.769934	928.275721	3.791962e+04	0.540239	0.087193	0.766259	0.664151	1.175033	8
min	6.762810e+09	1.000000	0.500000	370.000000	5.200000e+02	1.000000	0.000000	0.000000	1.000000	4.000000	3
25%	6.762815e+09	3.000000	1.750000	1440.000000	5.010750e+03	1.000000	0.000000	0.000000	3.000000	7.000000	12
50%	6.762821e+09	3.000000	2.250000	1930.000000	7.620000e+03	1.500000	0.000000	0.000000	3.000000	7.000000	15
75%	6.762826e+09	4.000000	2.500000	2570.000000	1.080000e+04	2.000000	0.000000	0.000000	4.000000	8.000000	22
max	6.762832e+09	33.000000	8.000000	13540.000000	1.074218e+06	3.500000	1.000000	4.000000	5.000000	13.000000	94

CORRELATION ANALYSIS

Correlation between numerical features and Price was calculated using `.corr()`. The top 5 features with highest positive correlation with Price were identified:

FEATURE	VALUE
Living Area	0.712
Grade	0.672
Area of House Excluding Basement	0.615
Number of Bathrooms	0.532
Living area renov	0.585

Linear Regression Results

Model trained on 5 high-correlation features. R^2 score on test data = 0.5627. This means model explains 56.27% of the total price variation and remaining 43.73% remains unexplained.

MNIST Clustering Analysis

K-Means with 10 clusters was applied on MNIST digits data without using labels.

Question 11: K-Means Clustering (5 Marks)

Import `KMeans` from `sklearn.cluster`. Initialize and fit the K-Means clustering model on the MNIST features (`X`). Since we know there are 10 digits (0-9), set the number of clusters to 10. Assign the cluster labels to a variable `kmeans_labels`.

Expected Output: A successfully fitted K-Means model and the cluster labels array.

```
1: # Write your answer here
from sklearn.datasets import fetch_openml
mnist = fetch_openml('mnist_784', version=1, as_frame=False, parser='auto')
X= mnist.data[:10000]
y= mnist.target[:10000].astype(int)
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=10, random_state=42, n_init=10)
kmeans.fit(X)
kmeans_labels = kmeans.labels_
print(kmeans)
print(kmeans_labels[:20])
```

```
KMeans(n_clusters=10, n_init=10, random_state=42)
[8 3 9 4 1 5 2 8 2 0 8 4 6 7 2 0 5 8 7 0]
```

Question 12: F1 Score Evaluation for Clustering (5 Marks)

Evaluate the clustering performance using the F1 score. Since K-Means assigns arbitrary cluster labels, you will first need to map each cluster label to the most frequent true label in that cluster. After mapping the labels, calculate and print the macro-averaged F1 score using `sklearn.metrics.f1_score`.

Expected Output: The calculated F1 score of the clustering.

```
# Write your answer here
from sklearn.metrics import f1_score
import numpy as np

y_pred = np.zeros_like(kmeans_labels)

for cluster in range(10):
    cluster_indices = (kmeans_labels == cluster)
    most_frequent_label = np.bincount(y[cluster_indices]).argmax()
    y_pred[cluster_indices] = most_frequent_label

f1 = f1_score(y, y_pred, average='macro')
print("F1 score:", f1)
```

CONFUSION MATRIX

To evaluate clustering performance, a confusion matrix was generated by comparing mapped cluster labels with true digit labels using `sklearn.metrics.confusion_matrix`. The matrix was converted to a Pandas DataFrame for better readability, with rows representing actual digits 0-9 and columns representing predicted cluster labels. From the 10x10 matrix, the diagonal values indicate correctly clustered digits. High values on the diagonal for digits like (1 = 1117 and 6 = 871) show that K-Means clustered these digits accurately. However, digits 3, 5, and 8 show high values in off-diagonal cells. For example, actual digit 3 was often predicted as cluster 8, and actual digit 5 was mixed with 3 and 8.

This confusion occurs because K-Means uses distance-based grouping and cannot distinguish visually similar digits without label information. As a result, the macro F1 score was 0.5100, indicating moderate clustering accuracy.

Generate and display the confusion matrix comparing your mapped cluster labels (from Question 9) to the true labels (`y`). Use `confusion_matrix` from `sklearn.metrics`. This helps evaluate how well your clusters align with the actual digit classes without requiring visual plotting.

Expected Output: A 10x10 confusion matrix printed as an array or a Pandas DataFrame.

```
In [ ]: # Write your answer here
from sklearn.metrics import confusion_matrix
import pandas as pd
cm = confusion_matrix(y, y_pred, labels=range(10))
df_cm = pd.DataFrame(cm, index=range(10), columns=range(10))
print(df_cm)
```

	0	1	2	3	4	5	6	7	8	9
0	783	2	5	45	19	0	37	7	103	0
1	0	1117	2	1	1	0	0	4	2	0
2	7	166	669	59	27	0	26	11	26	0
3	6	101	28	589	15	0	9	34	250	0
4	0	54	4	0	377	0	13	532	0	0
5	9	74	2	228	35	0	18	251	246	0
6	14	100	8	5	93	0	780	0	14	0
7	1	76	3	1	118	0	0	871	0	0
8	3	118	7	199	33	0	19	70	495	0
9	5	26	0	16	254	0	3	673	1	0

6. CONCLUSION

From this project, it was found that house price mainly depends on living area and grade of the house. The correlation analysis showed living area had 0.702 and grade of the house had 0.667 correlation with Price, which are the highest among all features. This proves that bigger houses and higher grade construction cost more. The Linear Regression model trained on 5 selected features gave an R^2 score of 0.5627 on test data. This means the model was able to explain 56.27% of the variation in house prices. So, the model works as a decent baseline, but 43.73% variation is still unexplained due to other factors like location, condition, etc. not included in the model. For the MNIST clustering task, K-Means with 10 clusters gave a macro F1 score of 0.5100. The confusion matrix showed digits like 0, 1, and 6 were clustered correctly, but digits like 5 and 8 got mixed up. This shows K-Means can separate data without labels, but accuracy is limited compared to supervised methods.

7. APPENDICES

Github link of the notebook: (https://github.com/SaptaparnaRoy/Internship-project/blob/main/05_Data_analysis_with_house_sales_data_Summer_2026/05_Data_analysis_with_house_sales_data_Summer_2026.ipynb)

REFERENCES:

- <https://www.hackerrank.com> (for solving questions)

